

- **Sensor Links (`lidar_link`, `camera_link`, `imu_link`):** The physical mounting points of your hardware, attached as children to the `base_link`.

4.1.2 Part 2: Real-world Autonomous Racing Use Cases

Static Broadcasters:

- **The LiDAR Mount:** The distance between the physical LiDAR sensor (`lidar_link`) and the center of the car (`base_link`) is defined by a metal bracket. It never, ever changes. We use a Static Broadcaster for this. TF2 will broadcast it once, and subscribers will cache it forever, saving massive network bandwidth.

Dynamic Broadcasters:

- **The Wheel Odometry:** As the car drives around the track, the position of the car (`base_link`) relative to its starting point (`odom`) changes 50 times a second. We use a Dynamic Broadcaster attached to the motor encoders to continuously update this mathematical relationship on the network.

4.1.3 Part 3: Implementation - The Static Broadcaster

Because static transforms never change, we rarely write C++ code for them. We launch them directly via the CLI or Python Launch files using the pre-built `tf2_ros` tools.

Via CLI (For quick track testing):

Terminal

```
# Syntax: X Y Z Yaw Pitch Roll parent_frame child_frame
# Example: LiDAR is 1.5m forward, 0.0m left, 1.0m high. No rotation.
ros2 run tf2_ros static_transform_publisher 1.5 0.0 1.0 0.0 0.0 0.0
base_link lidar_link
```

Via Python Launch File (For Production):

```
1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3
4 def generate_launch_description():
5     return LaunchDescription([
6         Node(
7             package='tf2_ros',
8             executable='static_transform_publisher',
9             name='lidar_static_tf',
10            # X, Y, Z, Yaw, Pitch, Roll, Parent, Child
11            arguments=['1.5', '0.0', '1.0', '0', '0', '0', '
12                base_link', 'lidar_link']
13        )
14    ])
```

4.1.4 Part 4: Implementation - Dynamic Broadcaster and Listener

We will write a C++ and Python node that acts as our Odometry system. It will dynamically broadcast the car moving forward. Then, we will write a Listener to read that movement.

1. The Dynamic Broadcaster

Modern C++ 14/17 Implementation

File: *src/odom_tf_broadcaster.cpp*

```

1  #include <memory>
2  #include "rclcpp/rclcpp.hpp"
3  #include "tf2_ros/transform_broadcaster.h"
4  #include "geometry_msgs/msg/transform_stamped.hpp"
5  #include "tf2/LinearMath/Quaternion.h" // For Euler to Quaternion
    conversion
6
7  using namespace std::chrono_literals;
8
9  class DynamicTFBroadcaster : public rclcpp::Node
10 {
11 public:
12     DynamicTFBroadcaster() : Node("odom_tf_broadcaster"), x_pos_(0.0)
13     {
14         // 1. Initialize the Broadcaster
15         tf_broadcaster_ = std::make_unique<tf2_ros::
            TransformBroadcaster>(*this);
16
17         // 2. Broadcast at 50Hz
18         timer_ = this->create_wall_timer(
19             20ms, std::bind(&DynamicTFBroadcaster::
                broadcast_timer_callback, this));
20     }
21
22 private:
23     void broadcast_timer_callback()
24     {
25         // 3. Create the Transform Message
26         geometry_msgs::msg::TransformStamped t;
27
28         // 4. Time stamp and frame IDs are CRITICAL
29         t.header.stamp = this->get_clock()->now();
30         t.header.frame_id = "odom"; // Parent
31         t.child_frame_id = "base_link"; // Child
32
33         // 5. Populate Translation (Simulating moving forward on
            X axis)
34         t.transform.translation.x = x_pos_;
35         t.transform.translation.y = 0.0;

```

```

36         t.transform.translation.z = 0.0;
37
38         // 6. Populate Rotation using Quaternions (Math
39           representation of 3D rotation)
40         tf2::Quaternion q;
41         q.setRPY(0.0, 0.0, 0.0); // Roll, Pitch, Yaw (0 radians =
42           straight ahead)
43         t.transform.rotation.x = q.x();
44         t.transform.rotation.y = q.y();
45         t.transform.rotation.z = q.z();
46         t.transform.rotation.w = q.w();
47
48         // 7. Send the transform to the TF tree
49         tf_broadcaster_ -> sendTransform(t);
50
51         x_pos_ += 0.01; // Move forward slightly every tick
52     }
53
54     std::unique_ptr<tf2_ros::TransformBroadcaster>
55         tf_broadcaster_;
56     rclcpp::TimerBase::SharedPtr timer_;
57     double x_pos_;
58 };
59
60 int main(int argc, char** argv) {
61     rclcpp::init(argc, argv);
62     rclcpp::spin(std::make_shared<DynamicTFBroadcaster>());
63     rclcpp::shutdown();
64     return 0;
65 }

```

Python Implementation

File: `odom_tf_broadcaster.py`

```

1  import math
2  import rclpy
3  from rclpy.node import Node
4  from tf2_ros import TransformBroadcaster
5  from geometry_msgs.msg import TransformStamped
6
7  def euler_to_quaternion(roll, pitch, yaw):
8      # Quick math helper: ROS 2 handles rotations exclusively in
9      Quaternions
10     qx = math.sin(roll/2) * math.cos(pitch/2) * math.cos(yaw/2) -
11         math.cos(roll/2) * math.sin(pitch/2) * math.sin(yaw/2)
12     qy = math.cos(roll/2) * math.sin(pitch/2) * math.cos(yaw/2) +
13         math.sin(roll/2) * math.cos(pitch/2) * math.sin(yaw/2)
14     qz = math.cos(roll/2) * math.cos(pitch/2) * math.sin(yaw/2) -
15         math.sin(roll/2) * math.sin(pitch/2) * math.cos(yaw/2)
16     qw = math.cos(roll/2) * math.cos(pitch/2) * math.cos(yaw/2) +
17         math.sin(roll/2) * math.sin(pitch/2) * math.sin(yaw/2)
18     return [qx, qy, qz, qw]

```

```

14
15 class DynamicTFBroadcaster(Node):
16     def __init__(self):
17         super().__init__('odom_tf_broadcaster')
18         self.tf_broadcaster = TransformBroadcaster(self)
19         self.x_pos_ = 0.0
20         self.timer = self.create_timer(0.02, self.
                broadcast_timer_callback) # 50Hz
21
22     def broadcast_timer_callback(self):
23         t = TransformStamped()
24
25         t.header.stamp = self.get_clock().now().to_msg()
26         t.header.frame_id = 'odom'
27         t.child_frame_id = 'base_link'
28
29         t.transform.translation.x = self.x_pos_
30         t.transform.translation.y = 0.0
31         t.transform.translation.z = 0.0
32
33         q = euler_to_quaternion(0.0, 0.0, 0.0)
34         t.transform.rotation.x = q[0]
35         t.transform.rotation.y = q[1]
36         t.transform.rotation.z = q[2]
37         t.transform.rotation.w = q[3]
38
39         self.tf_broadcaster.sendTransform(t)
40         self.x_pos_ += 0.01
41
42     def main():
43         rclpy.init()
44         rclpy.spin(DynamicTFBroadcaster())
45         rclpy.shutdown()
46
47 if __name__ == '__main__':
48     main()

```

2. The TF Listener

The Listener uses a Buffer to store the last 10 seconds of the TF tree, and queries it dynamically.

Modern C++ 14/17 Implementation Snippets

```

1 // Includes needed:
2 #include "tf2_ros/transform_listener.h"
3 #include "tf2_ros/buffer.h"
4
5 // Inside your node's constructor:
6 tf_buffer_ = std::make_unique<tf2_ros::Buffer>(this->get_clock())
    ;

```

```

7  tf_listener_ = std::make_shared<tf2_ros::TransformListener>(*
    tf_buffer_);
8
9  // In your timer loop to check where the car is:
10 try {
11     // "Look up the transform FROM odom TO base_link at the
        latest available time"
12     geometry_msgs::msg::TransformStamped t = tf_buffer_->
        lookupTransform(
13         "odom", "base_link", tf2::TimePointZero);
14     RCLCPP_INFO(this->get_logger(), "Car X Position: %.2f", t.
        transform.translation.x);
15 } catch (const tf2::TransformException & ex) {
16     RCLCPP_WARN(this->get_logger(), "Could not transform: %s", ex
        .what());
17 }

```

Python Implementation Snippets

```

1  # Imports needed:
2  from tf2_ros.buffer import Buffer
3  from tf2_ros.transform_listener import TransformListener
4
5  # Inside __init__:
6  self.tf_buffer = Buffer()
7  self.tf_listener = TransformListener(self.tf_buffer, self)
8
9  # In your timer callback:
10 try:
11     t = self.tf_buffer.lookup_transform('odom', 'base_link',
        rclpy.time.Time())
12     self.get_logger().info(f"Car X Position: {t.transform.
        translation.x}")
13 except Exception as e:
14     self.get_logger().warning(f"Could not transform: {e}")

```

4.1.5 Part 5: CLI Tools & Debugging (CRITICAL)

When a node crashes because it "Cannot transform lidar_link to map", your engineers will be blind unless they know how to inspect the tree.

1. The Wiretap: tf2_echo

You need to know the exact distance and math between two frames right now.

Terminal

```
# Syntax: ros2 run tf2_ros tf2_echo <target_frame> <source_frame>
ros2 run tf2_ros tf2_echo odom base_link

# Expected Terminal Output:
# At time 1708214532.123456
# - Translation: [1.250, 0.000, 0.000]
# - Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]
```

2. The Blueprint: view_frames (The Most Important Tool)

If `tf2_echo` fails, it means your tree is broken. A parent is missing, or a child is detached. You need to see the entire graph visually.

Terminal

```
# Navigate to a directory where you want to save the PDF
cd ~/ros2_ws

ros2 run tf2_tools view_frames

# Expected Terminal Output:
# Listening to tf data during 5 seconds...
# Generating graph in frames.pdf...
```

Tech Lead Note:

You open `frames.pdf`. It visually maps every active coordinate frame, who its parent is, who is broadcasting it, and the average broadcast frequency. If `lidar_link` is floating alone without an arrow connecting it to `base_link`, you instantly know your static transform publisher from Part 3 crashed.

CHAPTER 5

Phase 5: Advanced Node Architecture

5.1 Module 5.1: Composition (Component Nodes)

5.1.1 Part 1: The "Why" & The "How" (IPC & Zero-Copy)

The "Why"

If you launch a Camera Driver Node and a Lane Detection Node using standard `ros2 run` commands, the OS places them in two completely separate physical processes. They cannot see each other's memory.

To send a 15-Megabyte image from the camera to the detector, ROS 2 must:

1. Copy the image from Node A's memory to the DDS middleware.
2. Serialize it into a byte stream.
3. Push it through the OS loopback network interface (Inter-Process Communication, or IPC).
4. Deserialize it.
5. Copy it into Node B's memory.

Doing this at 30 or 60 Frames Per Second will max out your CPU before your perception algorithm even begins calculating.

The "How" (Architecture Level)

Composition completely eliminates this pipeline using Intra-Process Communication (Zero-Copy).

Instead of compiling your nodes as standalone executables (`.exe` / ELF binaries), you compile them as dynamically loadable Shared Libraries (`.so`).

You then launch a single, master OS process called a **Component Container**. You dynamically load both the Camera Node and the Perception Node into this exact same container. Because they now share the same physical memory space, ROS 2 bypasses the DDS